

0.100.1. Une lettre qui tire — chaque tir dans son sens

Le 0.100, avec trois tirs : chacun recopie le sens du A au moment où il part, dans `tdx[i]` et `tdy[i]`. Tire à droite, tourne-toi, tire en haut : deux tirs, deux sens, en même temps.

1. Ce qu'il faut comprendre

C'est le 0.100, avec plusieurs tirs. Le A tire dans le sens où il regarde, et le bouton A lance une rafale. Chaque tir garde **son** sens : tire à droite, tourne-toi vers le haut, tire encore, et les deux tirs s'éloignent chacun de leur côté.

Ce qui est nouveau ici : `tdx[3]` et `tdy[3]`. Au 0.100, le tir avait son sens, `tdx` et `tdy`. Maintenant, chaque tir a le sien : au départ du tir n° `i`, `tdx[i] = dx` et `tdy[i] = dy` recopient le regard du A. Ensuite, son arrivée est `tx[i] + tdx[i]`, `ty[i] + tdy[i]`.

Cinq tableaux pour trois tirs : `tx`, `ty`, `tdx`, `tdy`, `vole`. Tout ce qui décrit **un** tir est à la même place `i` dans les cinq. Au 0.102.1, on les rangera ensemble, dans une `struct`.

Déroulons : A en (2, 8), regard à droite. Bouton A : le tir 0 part en (3, 8), `tdx[0] = 1`, `tdy[0] = 0`. HAUT : le A monte en (2, 7) et regarde en haut. Bouton A : le tir 1 part en (2, 6), `tdx[1] = 0`, `tdy[1] = 255`. Le tir 0 continue vers la droite, le tir 1 vers le haut.

Essaie : tourne sur toi-même en gardant A enfoncé : les tirs partent dans quatre sens.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **999 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          73 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur()  124 o ← main() — une fonction fournie par le langage
main()             581 o ← démarrage — ta fonction
VBlank              8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
```

AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	– attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer(), texte()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	32 o	← CopierTuiles	– 2 lettres sur 43
"."	1 o	← texte()	– un texte gravé
"."	1 o	← texte()	– un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	68 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 2	46 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 2	26 o sur place, dans main()	+ AdresseCase (10 o)
bouton()	× 5	25 o sur place, dans main()	+ LireManette (37 o)
lire()	× 1	11 o sur place, dans main()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 3	32 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : dans le sens où le A regarde – plusieurs tirs ----
// Le 0.100 (le tir part dans le sens du regard), avec trois tirs.
// Chaque tir a SON sens : deux tableaux de plus, tdx et tdy.
```

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <lire>        // lit la tuile posée sur une case
#include <effacer>     // efface des cases, ou tout le fond
#include <texte>       // écrit un texte à l'écran
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;          // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;          // le sens où regarde le A (le 0.100)
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;          // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UN TABLEAU — TX

```
uint8_t tx[3];           // les trois tirs : leurs colonnes...
```

9. UN TABLEAU — TY

```
uint8_t ty[3];           // ...leurs lignes...
```

10. UN TABLEAU — TDX

```
uint8_t tdx[3];           // NOUVEAU : ...et leurs sens. Chacun garde le sien.
```

11. UN TABLEAU — TDY

```
uint8_t tdy[3];
```

12. UN TABLEAU — VOLE

```
uint8_t vole[3];           // vole[i] = 1 : le tir n° i est en vol
```

13. UNE VARIABLE — LANCE

```
uint8_t lance = 0;
```

14. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde (le 0.100) ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- le bouton A lance un tir dans le sens du regard, toutes les 250 ms au plus ----
        if (chaque(250)) {
            if (bouton(A)) {
                nx = ax + dx;
                ny = ay + dy;
                if (nx < 20 && ny < 17) {
                    lance = 0;
                    for (uint8_t i = 0; i < 3; i++) {
                        if (lance == 0 && vole[i] == 0) {
                            vole[i] = 1;
                            tx[i] = nx;
                            ty[i] = ny;
                            tdx[i] = dx;          // NOUVEAU : CE tir recopie le sens du A
                            tdy[i] = dy;
                            texte(tx[i], ty[i], ".");
                            lance = 1;
                        }
                    }
                }
            }
        }

        // ---- chaque tir avance dans SON sens ----
        // Tire à droite, tourne-toi, tire en haut : deux tirs, deux sens.
        if (chaque(100)) {
            for (uint8_t i = 0; i < 3; i++) {
                if (vole[i] == 1) {
```

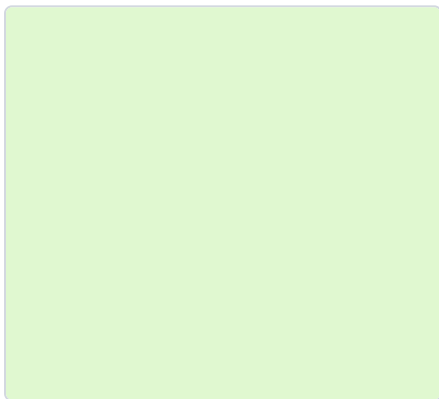
```

    nx = tx[i] + tdx[i];
    ny = ty[i] + tdy[i];
    effacer(tx[i], ty[i], 1);
    if (nx >= 20 || ny >= 17) {
        vole[i] = 0;          // hors de l'écran : il disparaît
    } else {
        tx[i] = nx;
        ty[i] = ny;
        texte(tx[i], ty[i], ".");
    }
}
}
}
}
}
}

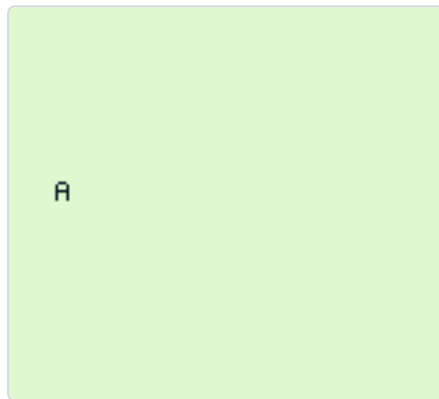
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



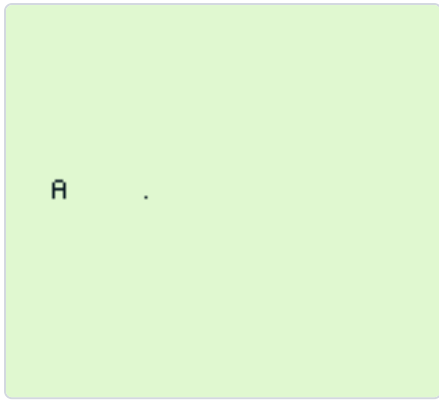
la toute première image — la console vient de démarrer — image 1



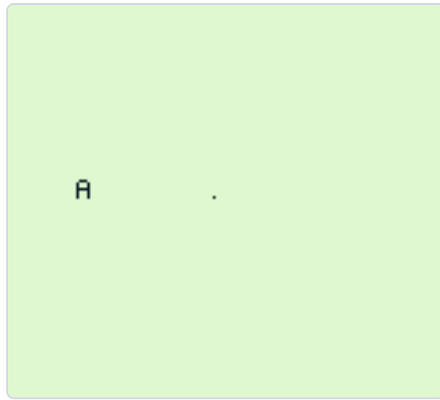
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

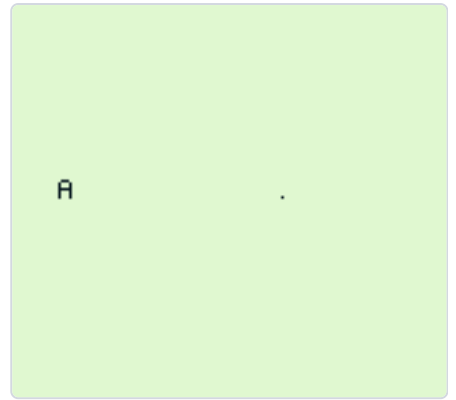
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



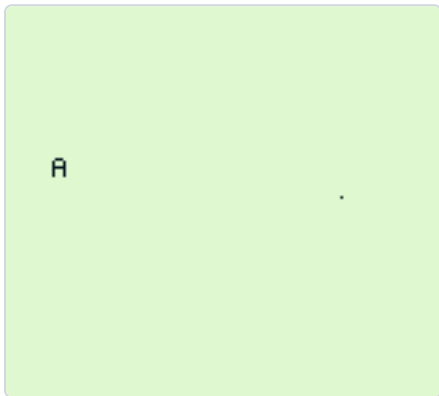
A — l'écran a changé



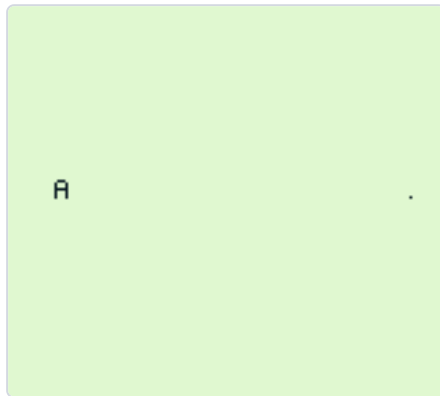
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A tire des rafales dans le sens où il regarde ; un tir lancé à droite continue à droite même après que le A s'est tourné pour tirer vers le haut.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ un tir à droite, puis le A se tourne vers le haut et tire encore
- ✓ deux tirs en même temps : un à droite sur la ligne 8, un au-dessus du A
- ✓ ils finissent par disparaître au bord

Ensuite : **0.101. Une lettre qui tire — un mur arrête le tir** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++